

Mini Project 2

Naomi Liftman

2026-02-21

Part A

Question 1

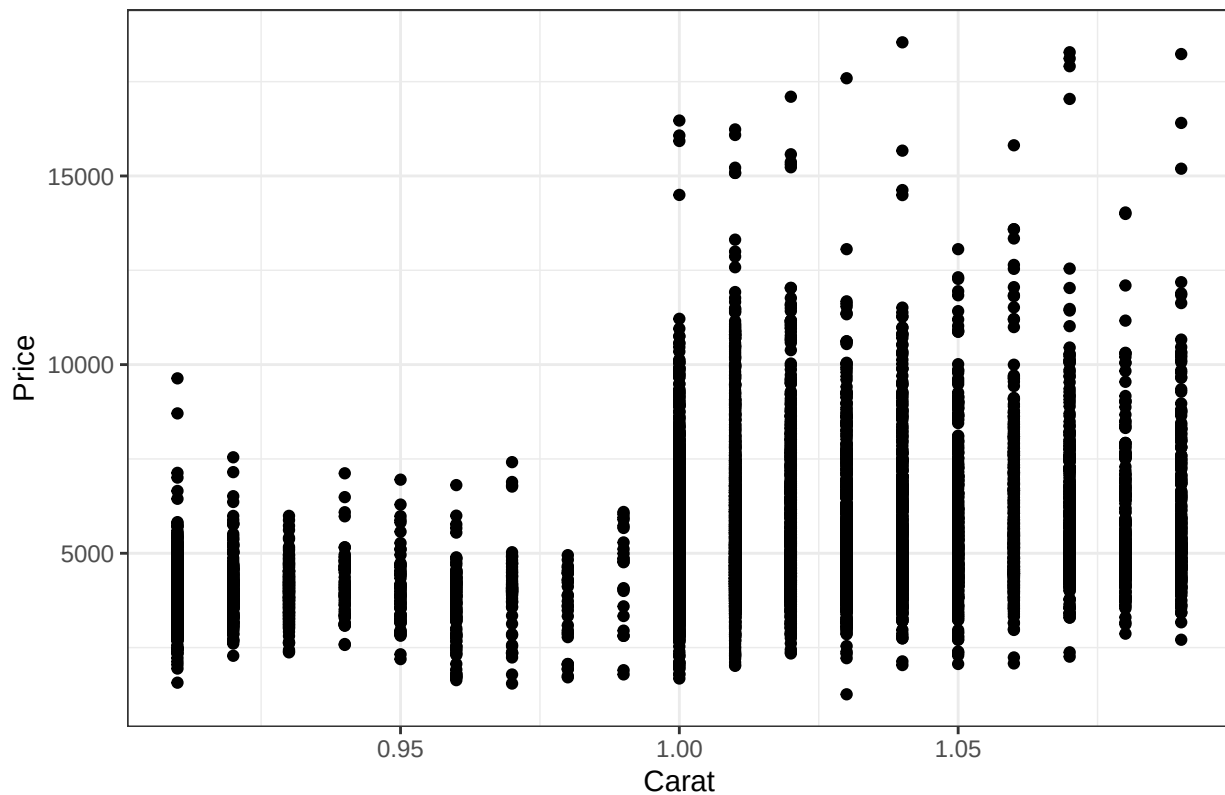
```
set.seed(10)
#load libraries and data
library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.2      v readr      2.1.4
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.4.2      v tibble    3.2.1
## v lubridate  1.9.2      v tidyr     1.3.0
## v purrr      1.0.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

library(ggplot2)
data(diamonds)
onlysmall <- diamonds %>%
  filter(carat > .9) %>%
  filter(carat < 1.1)

ggplot(data = onlysmall) +
  geom_point(aes(x = carat, y = price)) +
  labs(title = "Scatterplot of Diamond Price for Carats 0.9 - 1.1") +
  xlab("Carat") +
  ylab("Price") +
  theme_bw()
```

Scatterplot of Diamond Price for Carats 0.9 – 1.1



What we notice about this graph is that there is a very large increase in price between carats of .99 and carats of 1.0. We also see that there is a drastic change in the range of carats above and below the 1.0 threshold. Below 1.0 the range in price is between 2000 and 7500 with two outliers closer to 10000. However, the price range for diamonds above 1.0 carat is between 2000 and 17500.

Question 2.

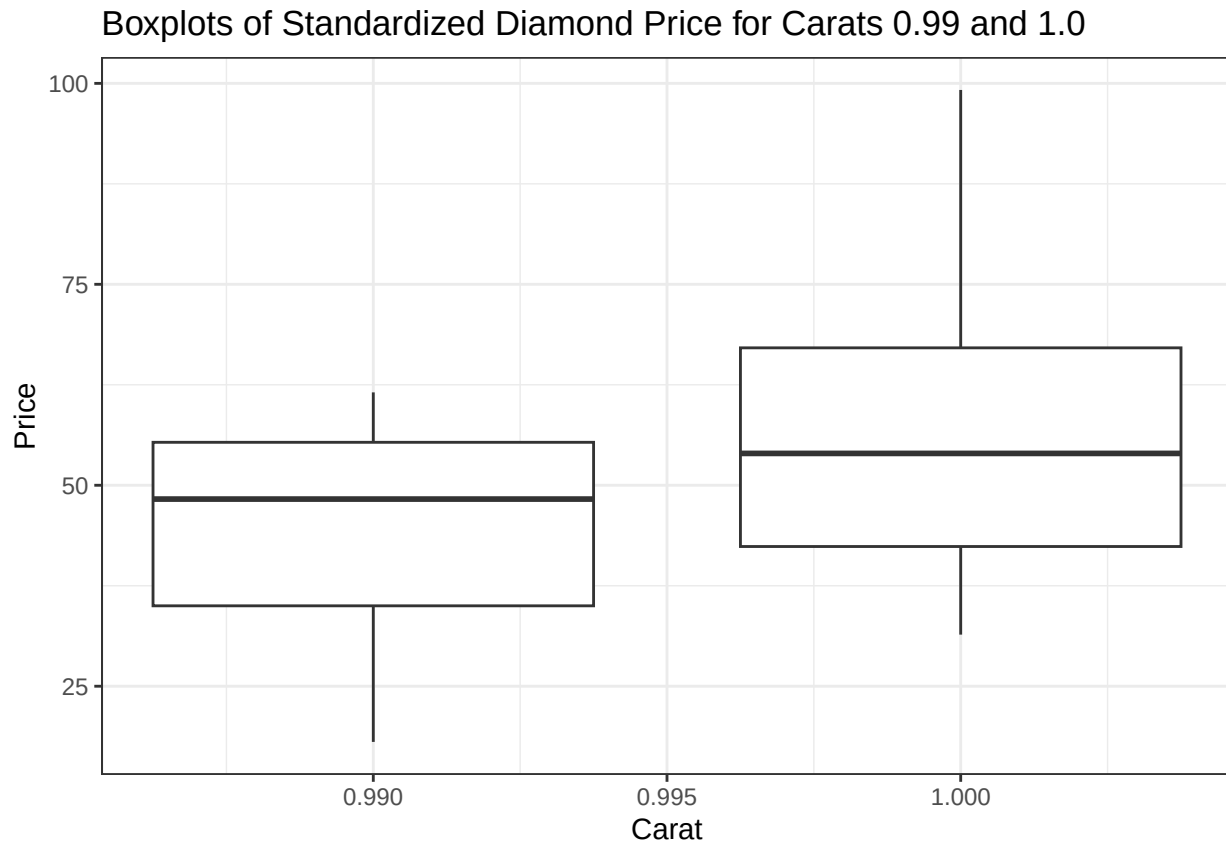
We standardized the prices because then we are able to see the price of a diamond per .01 carats. So then we can see the difference in price based on very small differences (.01 carat) in the diamonds.

Question 3.

```
#this is the setupcode and should only be run once or else will not be the same
set.seed(557)
diamonds_small <- diamonds %>%
filter(carat %in% c(0.99, 1.0)) %>%
group_by(carat) %>%
sample_n(23)
#standardized price
diamonds_small <- diamonds_small %>%
  mutate(std_price = price/(100*carat)) %>%
  group_by(carat)

ggplot(data = diamonds_small) +
```

```
geom_boxplot(aes(x = carat, y = std_price, group = carat)) +
labs(title = "Boxplots of Standardized Diamond Price for Carats 0.99 and 1.0") +
xlab("Carat") +
ylab("Price") +
theme_bw()
```



Graphically it appears that the medians are quite close to each other; however, the ranges are extremely different with the 1.00 range being much larger than the 0.99. Numerically we can see that the means are not very close to each other, with the mean for 1.00 carat being much larger than .99. We also note the standard deviations are drastically different with the standard deviation for 1.00 being larger than the standard deviation for .99.

```
diamonds_small %>%
  group_by(carat) %>%
  summarise(mean = (mean(std_price)), sd = sd(std_price))
```

```
## # A tibble: 2 x 3
##   carat  mean    sd
##   <dbl> <dbl> <dbl>
## 1  0.99  44.5  13.3
## 2    1   56.3  17.5
```

Question 4.

a. The null hypothesis is that there is no difference in standardized prices between 0.99 and 1 carat diamonds. The alternative hypothesis is that there is a difference in standardized prices between .99 and 1 carat diamonds.

b. We will be performing a Welch t-test for difference in two means. We will be using the Welch test because there is clearly a difference in the variances of each group and this test does not require assuming equal variances.

c. The distribution of the test statistic is a t-distribution (the df are slightly different for a Welch test, but R automatically calculates these and gets 41.093 for this model).

d. To find the rejection rule for this test we can look up a two-sided t-score chart for a Welch's t-test. We know when we run the test we get a degrees of freedom of 41.093, so we get a rejection value of -2.02 and +2.02. So our rejection region is a test statistic below -2.02 or above 2.02.

```
# Subset the data for automatic and manual transmission cars
smaller <- subset(diamonds_small, carat == 0.99)$std_price
larger <- subset(diamonds_small, carat == 1.00)$std_price

# Perform Welch's t-test (so we can find the degrees of freedom to put in table)
result <- t.test(smaller, larger, var.equal = FALSE)
result
```

```
##
## Welch Two Sample t-test
##
## data: smaller and larger
## t = -2.58, df = 41.093, p-value = 0.01355
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
## -21.091434 -2.570604
## sample estimates:
## mean of x mean of y
## 44.50681 56.33783
```

e. The value of our test-statistic -2.57997. We know that our rejection region is any t-score above 2.02 or below -2.02. Our test-statistic is below the cutoff value which means that we would reject the null hypothesis.

```
result$statistic
```

```
##          t
## -2.57997
```

f. Using a type 1 error rate of 10% we can again look at our Welch's t-score distribution again we get cutoff values of 1.68 and -1.68. Which means our rejection region is a test statistic that falls above 1.68 or below -1.68.

We still have the same test statistic so we will continue to reject the null hypothesis.

```

# Subset the data for automatic and manual transmission cars
smaller <- subset(diamonds_small, carat == 0.99)$std_price
larger <- subset(diamonds_small, carat == 1.00)$std_price

# Perform Welch's t-test
result <- t.test(smaller, larger, var.equal = FALSE)
result

```

g.

```

##
##  Welch Two Sample t-test
##
## data:  smaller and larger
## t = -2.58, df = 41.093, p-value = 0.01355
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  -21.091434  -2.570604
## sample estimates:
## mean of x mean of y
##  44.50681  56.33783

```

We get a p-value of 0.01355. This means that we reject the null hypothesis, which is consistent with our findings from part e. This means that we have statistically significant evidence to suggest that there is a difference between the average standardized price of diamonds of carat 0.99 versus 1.00.

Question 5

We find a confidence interval of (-21.091434, -2.570604). This means that we are 95% confident that the true difference between the means of diamonds of carat 0.99 and 1.00 is between (-21.091434, -2.570604).

```
result$conf.int
```

```

## [1] -21.091434  -2.570604
## attr(,"conf.level")
## [1] 0.95

```

Part B

Question 1

```

library(stats)
#create a df for the means
mean_dif <- data.frame(mean_99 = NA, mean_1 = NA)
#give the means based on shuffled data!
for (i in 1:1000){
  #take the transformed data

```

```

df_shuffled = transform(diamonds_small, shuffled_price = sample(std_price))
#just the df for 99
nine <- df_shuffled %>%
  filter(carat == 0.99)
one <- df_shuffled %>%
  filter(carat == 1.00)
mean_dif[nrow(mean_dif) + 1,] = c(mean(nine$shuffled_price), mean(one$shuffled_price))
}
#drop the first row
mean_dif <- data.frame(mean_dif[-1,])
#now get the difference!
mean_dif <- mean_dif %>%
  mutate(diff = mean_1 - mean_99)

#original sample difference
nine <- diamonds_small %>%
  filter(carat == 0.99)
one <- diamonds_small %>%
  filter(carat == 1.00)

dif = (mean(one$std_price) - mean(nine$std_price))

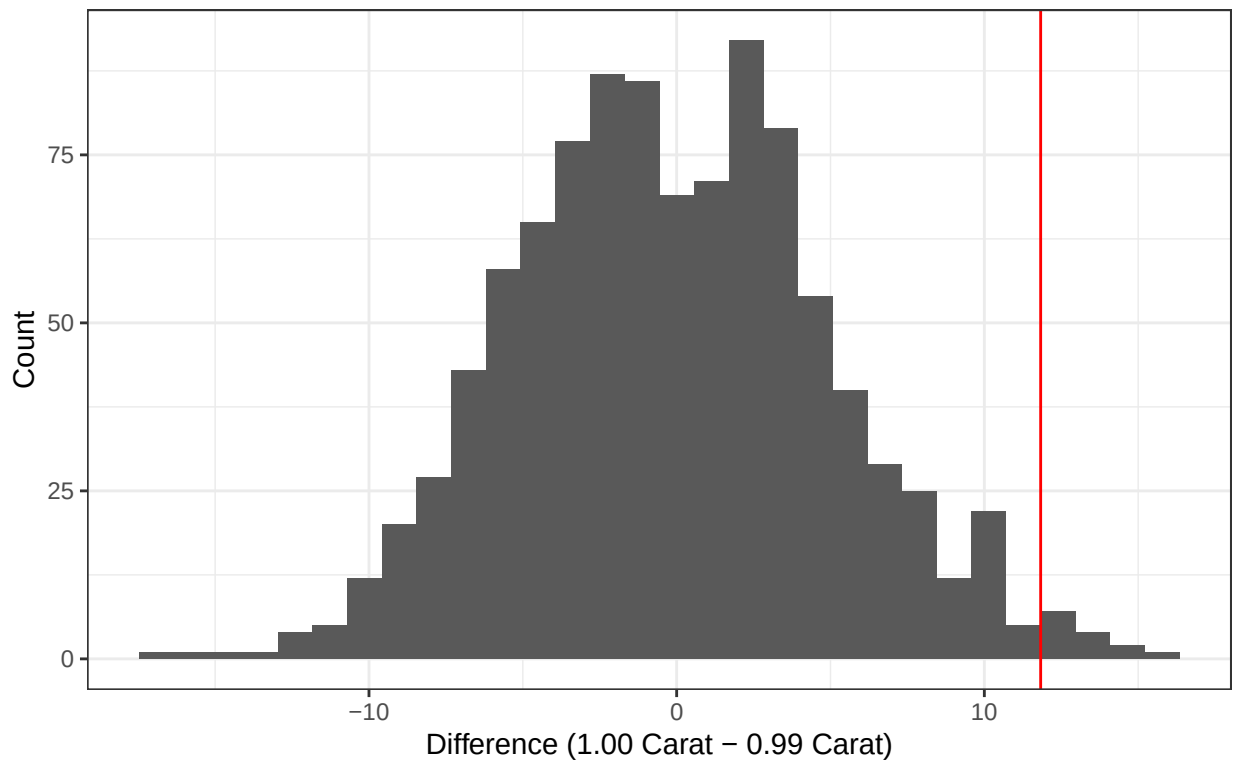
ggplot(data = mean_dif) +
  geom_histogram(aes(x = diff)) +
  labs(title = "Histogram of Difference in Means for Randomized Samples", caption = "*Red vertical line") +
  xlab("Difference (1.00 Carat - 0.99 Carat)") +
  ylab("Count") +
  theme_bw() +
  geom_vline(xintercept = 11.831, color = "red")

```

a.

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```

Histogram of Difference in Means for Randomized Samples



*Red vertical line denotes original sample difference in means

b. I calculated a one sided p-value of 0.014.

```
mean_dif %>%
  count(diff > 11.831)
```

```
##   diff > 11.831   n
## 1          FALSE 986
## 2           TRUE  14
```

```
14/1000
```

```
## [1] 0.014
```

c. Using the significance level of 5%, we conclude that we reject the null hypothesis. In this context it means that we have evidence suggest that the average standardized price values of diamonds 0.99 carats is smaller than diamonds of 1.0 carats.

Question 2

```

set.seed(730)
nine_small <- nine %>%
  select(carat, std_price)
one_small <- one %>%
  select(carat, std_price)

b <- 1000
mean_one <- c()
mean_nine <- c()
for (i in 1:b){
  one.resample <- sample(one_small$std_price, replace = TRUE)
  mean_one[i] <- mean(one.resample)
  nine.resample <- sample(nine_small$std_price, replace = TRUE)
  mean_nine[i] <- mean(nine.resample)
}

total_boots <- data.frame(one_mean = mean_one, nine_mean = mean_nine, dif = mean_one - mean_nine)

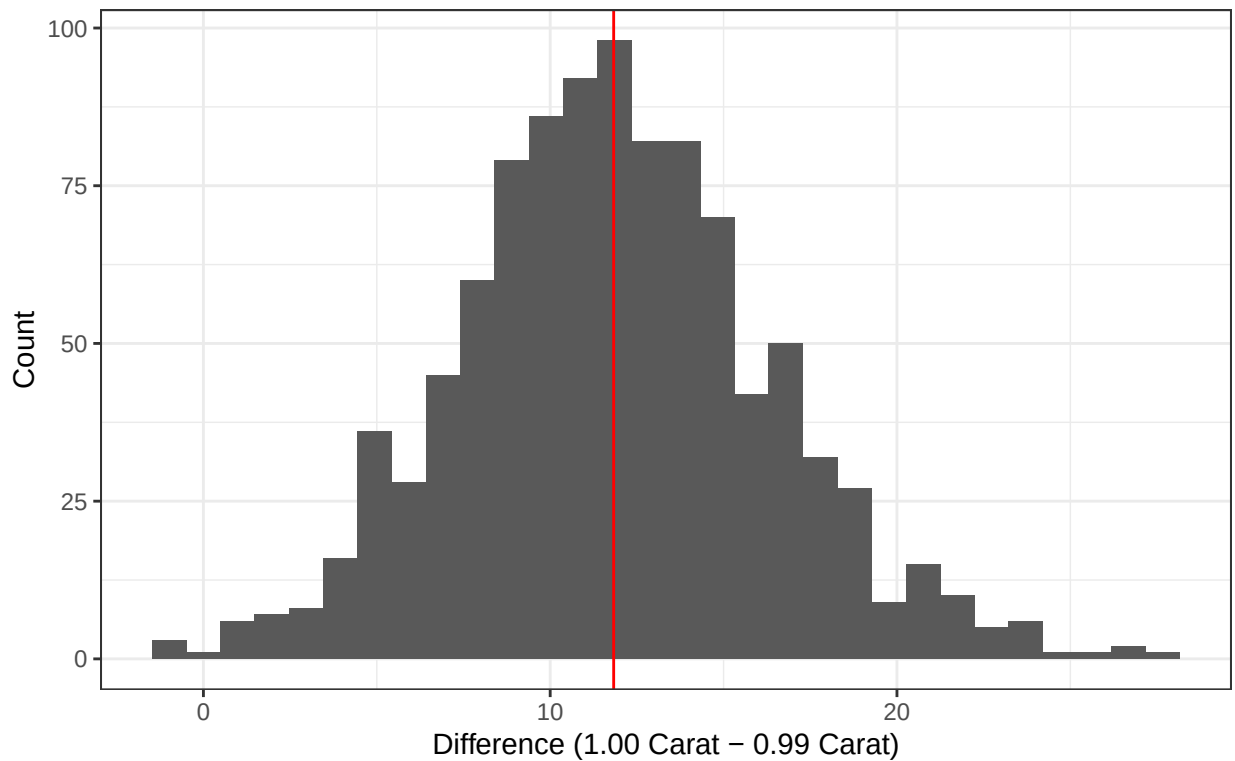
ggplot(data = total_boots) +
  geom_histogram(aes(x = dif)) +
  labs(title = "Histogram of Difference in Means for Bootstrapped Samples", caption = "*Red vertical line",
        xlab("Difference (1.00 Carat - 0.99 Carat)") +
        ylab("Count") +
        theme_bw() +
        geom_vline(xintercept = 11.831, color = "red"))

```

a.

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```


Histogram of Difference in Means for Bootstrapped Samples



*Red vertical line denotes original sample difference in means

```
#get quantile CI
quantile(total_boots$dif, probs = c(.025, .975))
```

b.

```
##      2.5%      97.5%
## 3.663001 21.397303
```

Based on our bootstrap distribution shown in the histogram above we get a 95% bootstrap percentile confidence interval of (3.663001, 21.397303).

c. The standard deviation of the bootstrap sample is 4.463296. We can then plug this into the confidence interval formula that was used in Question 5.

```
#sd
sd(total_boots$dif)
```

```
## [1] 4.463296
```

```
#original diff: 11.83102
#get the t-value based on a t-table and df = 46-2 = 44
#gives us 2.021
(11.83102 - 4.463296*2.021)
```

```
## [1] 2.810699
```

```
(11.83102 + 4.463296*2.021)
```

```
## [1] 20.85134
```

We get a confidence interval of (2.810699, 20.85134).

Part C

Previous Findings

Our findings from parts A and B is that we do have statistically significant evidence that there is a difference in standardized prices for diamonds of carats 0.99 and carats 1.0. In part A we used a Welch's t-test for difference of means when assuming different standard deviations and found that there was a statistically significant difference. In part B we used both randomization and bootstrapping. Randomization showed that there was a statistically significant difference in means and the bootstrapping method showed a confidence interval that did not include zero. This means that we are 95% confident that the true difference in population means is not zero.

Possible explanation is that there is truly a difference in standardized price means for the diamonds of carats 0.99 and carats 1.0. The other explanation is that our sample is completely skewed. It is possible that we got the only 46 diamonds out of the population (we are considering our large dataset of 54940 diamonds as our population) that would produce a difference in means for the two groups. While this is pretty unlikely, it is possible.

Carat 1.0 Cut, Clarity, and Color

I am going to start by gaining some descriptive statistics to understand the breakdown of our data.

```
#pull only 1 carat
only_one <- diamonds %>%
  filter(carat == 1.0) %>%
  mutate(price = as.numeric(price))

only_one %>%
  group_by(cut) %>%
  count(cut)
```

```
## # A tibble: 5 x 2
## # Groups:   cut [5]
##   cut          n
##   <ord>      <int>
## 1 Fair         166
## 2 Good         333
## 3 Very Good   389
## 4 Premium     462
## 5 Ideal       208
```

```
only_one %>%
  group_by(color) %>%
  count(color)
```

```
## # A tibble: 7 x 2
## # Groups:   color [7]
##   color      n
##   <ord> <int>
## 1 D      199
## 2 E      317
## 3 F      350
## 4 G      312
## 5 H      224
## 6 I       99
## 7 J       57
```

```
only_one %>%
  group_by(clarity) %>%
  count(clarity)
```

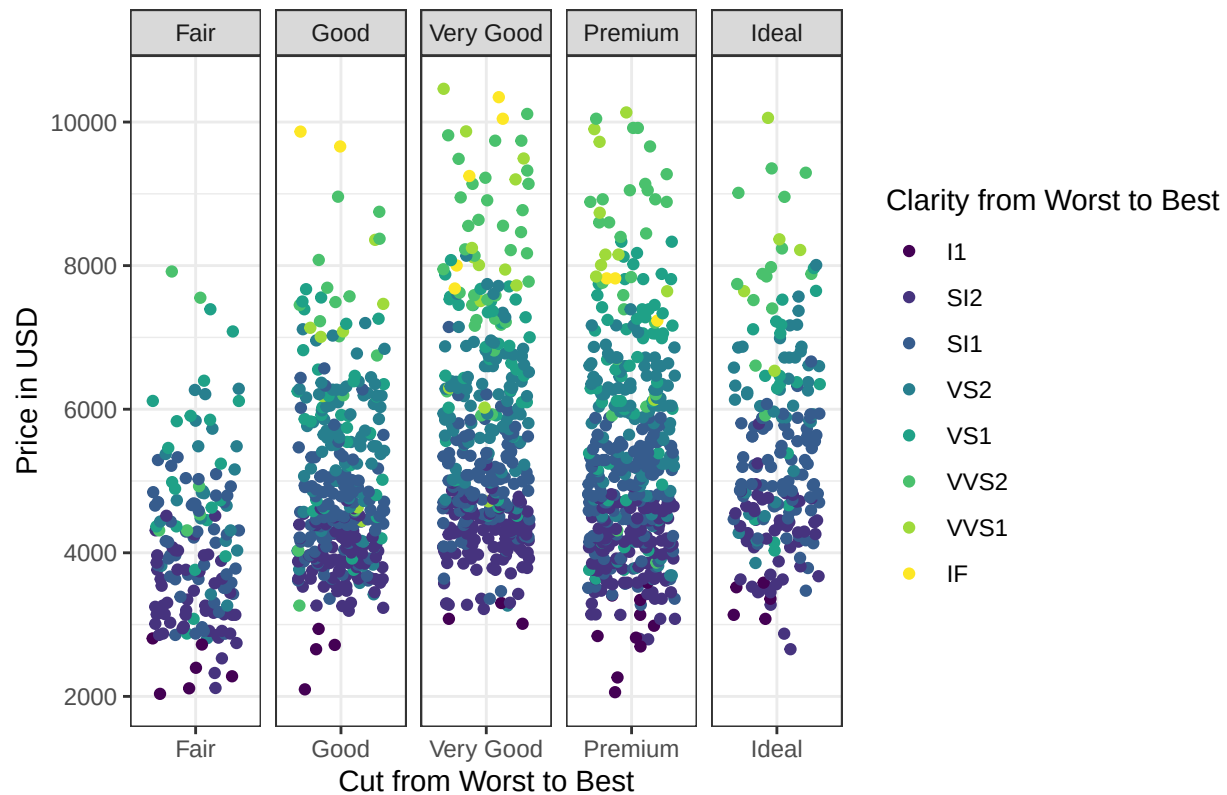
```
## # A tibble: 8 x 2
## # Groups:   clarity [8]
##   clarity      n
##   <ord>   <int>
## 1 I1        34
## 2 SI2       409
## 3 SI1       463
## 4 VS2       335
## 5 VS1       163
## 6 VVS2       100
## 7 VVS1        39
## 8 IF        15
```

First, I notice that there is a pretty big distribution in the amount of data we have within each category. For example for cut, we notice that we have 462 **premium** diamonds but only 166 **fair** diamonds. The same goes for color and clarity. For color we have 317 E but only 57 J diamonds, and clarity we have 34 I1 and 463 SI2 diamonds. This should be taken into account when looking at the graphs produced below.

```
#create a pretty plot!
ggplot(data = only_one) +
  geom_point(aes(x = cut, y = price, color = clarity), position = "jitter") +
  facet_grid(~cut, scales = "free_x") +
  scale_y_continuous(limits = c(2000, 10500), name = "Price in USD") +
  xlab(label = "Cut from Worst to Best") +
  labs(title = "Diamond Cut & Clarity's Effects on Price", color = "Clarity from Worst to Best") +
  theme_bw()
```

```
## Warning: Removed 16 rows containing missing values ('geom_point()').
```

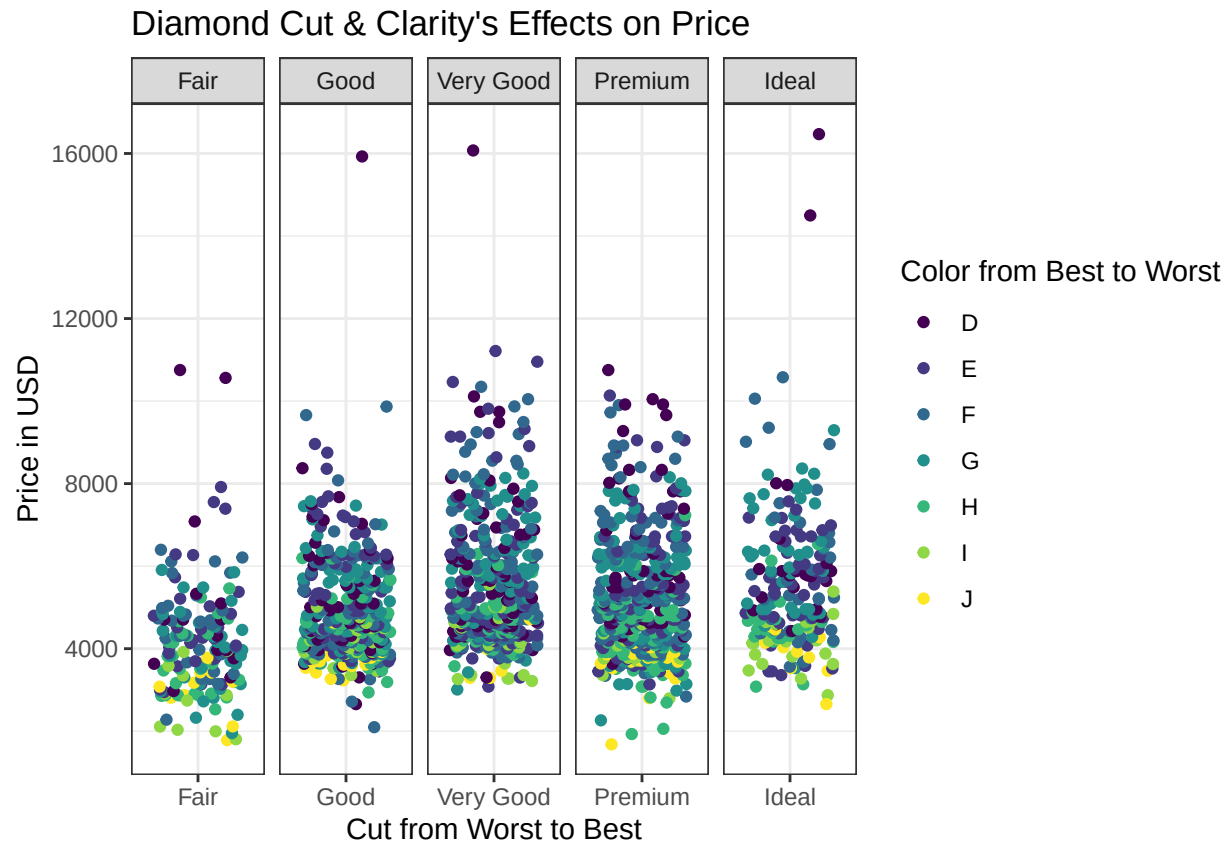
Diamond Cut & Clarity's Effects on Price



```
#things to know
#Fair, Good, Very Good, Premium, Idea
#diamond colour, from D (best) to J (worst)
#a measurement of how clear the diamond is (I1 (worst), SI2, SI1, VS2, VS1, VVS2, VVS1, IF (best))
```

Based on this graph we can see that as the clarity of diamond gets better the price increases. However, we do notice that for “top of the line” diamonds (or clarity VVS2 and above) there is less distinction around price. For example looking at the **Very Good** cut we can see that some IF diamonds are priced lower than some VVS1s and VVS2s. So it appears that as clarity gets better the price increases, to a point, with a platea above VS1 clarity.

```
ggplot(data = only_one) +
  geom_point(aes(x = cut, y = price, color = color), position = "jitter") +
  facet_grid(~cut, scales = "free_x") +
  scale_y_continuous(name = "Price in USD") +
  xlab(label = "Cut from Worst to Best") +
  labs(title = "Diamond Cut & Clarity's Effects on Price", color = "Color from Best to Worst") +
  theme_bw()
```

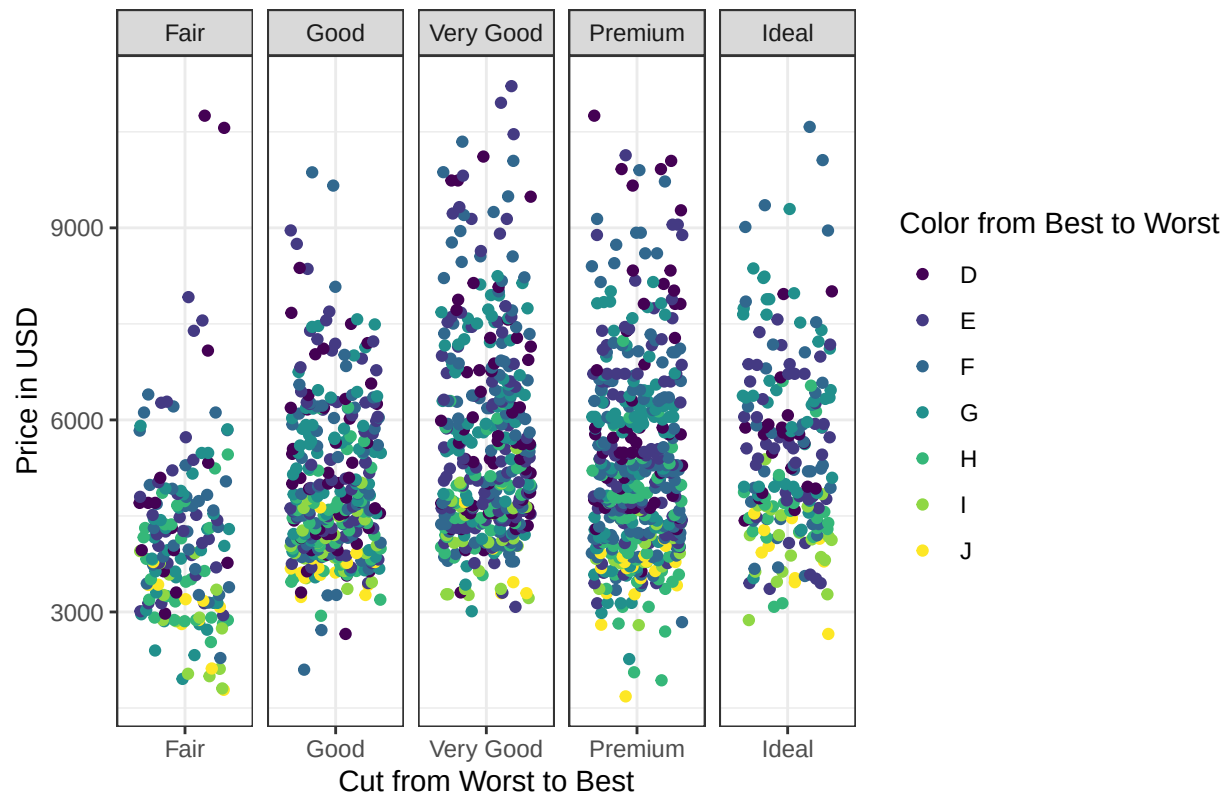


Now looking at this plot for color we can see four distinct outliers which I removed in the next plot to better understand the spread of color.

```
only_one_dropped <- (only_one[-(1533:1536),])

ggplot(data = only_one_dropped) +
  geom_point(aes(x = cut, y = price, color = color), position = "jitter") +
  facet_grid(~cut, scales = "free_x") +
  scale_y_continuous(name = "Price in USD") +
  xlab(label = "Cut from Worst to Best") +
  labs(title = "Diamond Cut & Clarity's Effects on Price", color = "Color from Best to Worst") +
  theme_bw()
```

Diamond Cut & Clarity's Effects on Price

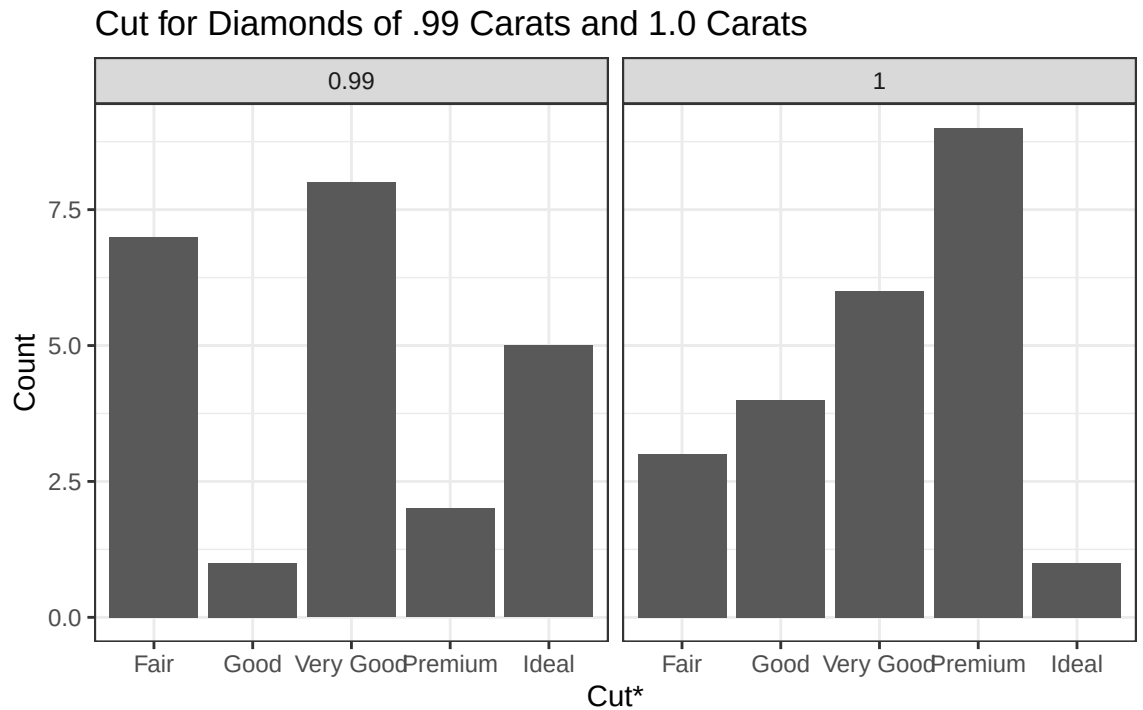


This graph gives us a much better idea of the effects of color on price. What we notice is that the color appears to have some effect on price, but not a ton. We can see that diamonds in the worst two colors (J and I) have some of the lowest prices; however, looking in the **Good** category we can see that a diamond with the best color D has one of the lowest prices.

What we find overall is that **Very Good** diamonds have some of the highest prices even without accounting for color and clarity. We also find that color has little effect on the price, but clarity has a large effect on the price.

All Diamonds

```
ggplot(data = diamonds_small) +
  geom_bar(aes(x = cut)) +
  facet_grid(~carat) +
  theme_bw() +
  labs(title = "Cut for Diamonds of .99 Carats and 1.0 Carats", caption = "*read left to right is worst",
        xlab("Cut*") +
        ylab("Count"))
```



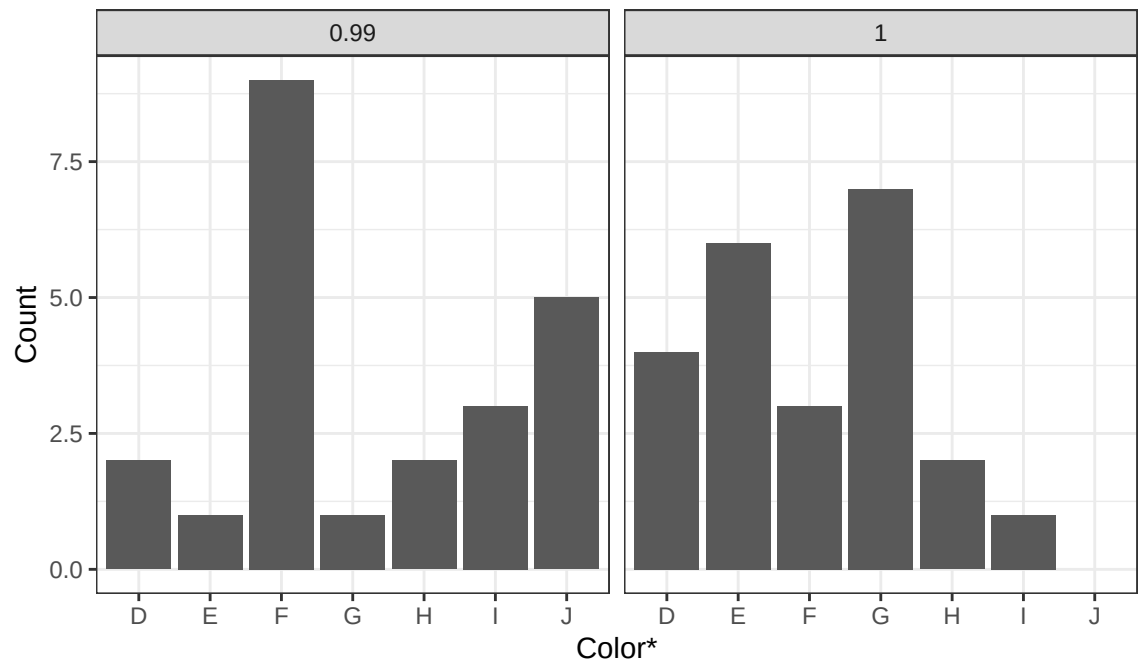
1. Carat & Cut

*read left to right is worst to best

We can see that the distributions of cut for each diamond are very different. What we can see is that for diamonds of carat 0.99 it is very unlikely for that diamond to be a **good** or **premium** cut. What we can see for diamonds of carat 1.00 is that they are very unlikely to be an **ideal** cut, but they are very likely to be **premium**. It appears that there are differences in the distributions in cuts based on carat; however, there does not appear to be any clear trends for diamonds of carat 0.99.

```
ggplot(data = diamonds_small) +
  geom_bar(aes(x = color)) +
  facet_grid(~carat) +
  theme_bw() +
  labs(title = "Color for Diamonds of .99 Carats and 1.0 Carats", caption = "*read left to right is best",
        xlab("Color*") +
        ylab("Count"))
```

Color for Diamonds of .99 Carats and 1.0 Carats

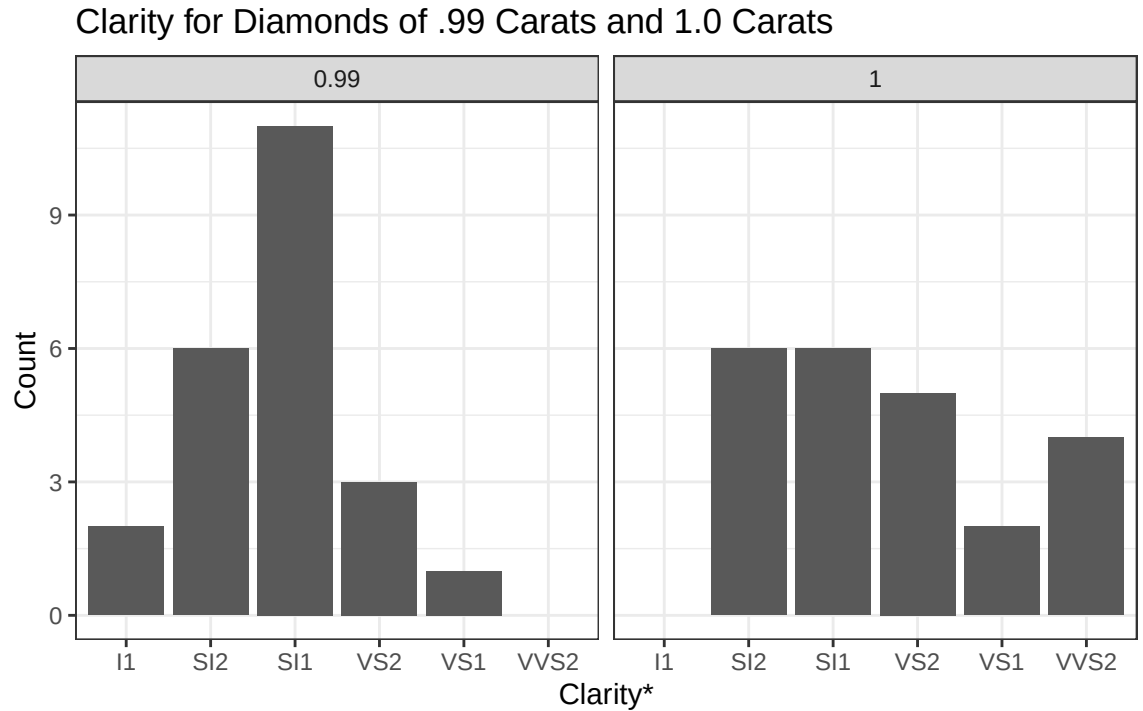


*read left to right is best to worst

2. Carat & Color

What we can see from our graph is that diamonds of carats 0.99 the most common color (by far) is F. This is in the middle of the range but leaning towards the best; however, the second most category is J, which is the worst color. We can see that for diamonds of carat 1.0 the distribution leans towards the best colors (D through G) and there are no diamonds in the worst category. So I would suggest that diamonds of carat 1.0 are more likely to have a better color than diamonds of carat 0.99.

```
ggplot(data = diamonds_small) +
  geom_bar(aes(x = clarity)) +
  facet_grid(~carat) +
  theme_bw() +
  labs(title = "Clarity for Diamonds of .99 Carats and 1.0 Carats", caption = "*read left to right is w",
        xlab("Clarity*") +
        ylab("Count"))
```

*read left to right is worst to best

3. Carat & Clarity

We can see that for diamonds of carat 0.99 the most common clarity is S11 which is middle of the road in terms of clarity. We can also see that there are very few diamonds with better clarity and no diamonds of best clarity across our sample of diamonds of carat 0.99. What we can see for diamonds of carat 1.0 there is a more even distribution (almost uniform) across clarity; however, there are no diamonds of carat 1.0 that have the worst clarity (I1). Thus I would conclude that diamonds of carat 1.0 are usually diamonds of a better clarity than diamonds of 0.99 carats.

Final Findings

What we found above is that a diamond being 1.0 versus 0.99 carats may be effected by the cut, clarity, and color. We can also see that these three variables may effect the non-standardized price of diamonds 1.00 carats. What this tells us is that we may want to use other variables (not just carat) to determine the differences between the two carats of diamonds. What we found is that there may have been confounding variables that effected our results in parts A and B!